MINI PROJECT REPORT

PARALLEL MATRIX MULTIPLICATION

Submitted by:
Kajal Mourya
1320725019

In partial fulfilment for the award of the degree of

MASTER OF TECHNOLOGY
In
COMPUTER SCIENCE AND ENGINEERING

Under the guidance of
Dr. Shelly Shiju George

Associate Professor, ASAC



Alliance School Of Advance Computing

Alliance University, Bengaluru

October-2025



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on Multicore Computing is submitted by Kajal Mourya [Uni. Roll No. 1320725019] in partial fulfilment for the award of the degree of Master of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026. This report embodies the results of original work and studies conducted by students and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

Dr. Shelly Shiju George
Associate Professor



COMPUTER SCIENCE AND ENGINEERING
ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled PARALLEL MATRIX MULTIPLICATION using OpenMP, submitted by me in partial fulfilment of the requirements for the award of the degree of **Master of Technology in Computer Science and Engineering (CSE)** at **Alliance University**, is a record of my original work carried out under the supervision of **Dr. Shelly Shiju George, Associate Professor**.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Kajal Mourya	1320725019	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled PARALLEL MATRIX MULTIPLICATION using OpenMP.

First and foremost, I extend my sincere thanks to my mentor and guide, **Dr. Shelly Shiju George**, for her unwavering support, insightful feedback, and encouragement throughout the development of this system. Her expertise in computer science and her commitment to fostering innovation provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to successfully compare the sequential and parallel execution time in matrix multiplication algorithms using OpenMP. It strengthened my belief in the power of technology to solve critical challenges in multicore computing.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my vision made this accomplishment possible.

Thank you.

Kajal Mourya

TABLE OF CONTENTS

Acknowledgement 4

Abstract 6

Chapter I - Introduction 7

1.1 Project Overview & Backend Context 7

1.2 Problem Statement in Server-Side Architecture 7

1.3 Project Objectives (Backend Scope) 7

Chapter II - Technologies & System Configuration 8

2.1 Technology Stack & Core Dependencies 8

2.2 Server Runtime & Hardware Environment 8

2.3 Layered Architectural Model (Routes, Middleware, Controllers) 9

Chapter III - API Design & Implementation 10

3.1 RESTful API Endpoint Contracts & Envelope Design 10

3.2 Authentication & Session Architecture (/api/auth) 10

3.3 Document CRUD Endpoints & Ownership Scoping 11

3.4 Pagination, Multi-Field Sorting & Case-Insensitive Search 11

3.5 Real-Time Category Aggregation Pipeline (/api/documents/stats) 12

3.6 Centralized Error Handling & Not Found Handlers 12

3.7 Security Protocols (CORS, Password Hashing, JWT Cookies) 12

Chapter IV - Testing Methodology & Test Cases 13

4.1 Backend Verification Strategy 13

4.2 Comprehensive Backend Test Cases 13

Chapter V - Results & Discussion 14

5.1 API Execution Metrics & Latency Profiling 14

5.2 Multitenant Security & Authorization Evaluation 14

Chapter VI - Conclusion & Future Enhancements 15

6.1 Conclusion 15

6.2 Future Enhancements 15

References 15

Abstract

Modern web applications demand robust, scalable, and secure backend architectures capable of serving high-throughput RESTful endpoints while enforcing strict data isolation across multitenant user bases. This report presents the architectural design, implementation, and empirical evaluation of the backend service for **FileHub – Smart File & Document Organizer**, developed using **Node.js**, **Express.js**, and **Prisma ORM**.

The backend system exposes two primary RESTful resource domains: an authentication engine (/api/auth) and a document management API (/api/documents). Authentication is implemented via stateless JSON Web Tokens (JWT) conveyed through HTTP-only, SameSite cookies, ensuring complete immunity against Cross-Site Scripting (XSS) token exfiltration. Passwords are salted and hashed using bcrypt with 12 computation rounds before database persistence. An extensible authentication middleware intercepts all incoming requests to protected routes, cryptographically validates token signatures, verifies active database presence, and injects authenticated user context into downstream request handlers.

A central requirement of the document service is the strict enforcement of private data ownership. Every document entity is tied to a user identifier in MongoDB. The document controller guarantees that all create, read, update, delete, and aggregation queries are unconditionally scoped to the authenticated user's ID, preventing horizontal privilege escalation attacks. Server-side pagination, regex-based case-insensitive name searching, whitelist sorting, and parallel category aggregation pipelines ensure consistent sub-3ms query latencies.

Empirical benchmarks demonstrate high concurrency resilience, with the server processing over 100 simultaneous client requests with an average latency below 16 milliseconds. Centralized error handling, standardized JSON response envelopes, and graceful process termination handlers establish FileHub as a production-ready, enterprise-grade backend service.

Chapter I - Introduction

1.1 Project Overview & Backend Context

In distributed web architectures, the server layer serves as the authoritative boundary for business logic enforcement, data access orchestration, cryptographic security, and state persistence. **FileHub** is designed as a centralized full-stack document organizer where users securely upload, categorize, filter, search, and manage critical files across academic, professional, and personal domains.

The backend service provides the computational engine for FileHub. Implemented as a modular Node.js and Express.js REST application, it interfaces directly with MongoDB through Prisma ORM. The service handles user identity management, password hashing, session issuance via HTTP-only cookies, structured CRUD operations, real-time statistical aggregations, input sanitization, and centralized exception recovery.

1.2 Problem Statement in Server-Side Architecture

Server-side systems for document management confront several critical architectural challenges:

- **Horizontal Privilege Escalation (Insecure Direct Object References - IDOR):** Applications that rely solely on frontend access controls or trust client-supplied user identifiers allow malicious actors to access, modify, or delete another user's documents simply by altering a document ID parameter.
- **Session Hijacking & Vulnerable Token Storage:** Storing authentication tokens in browser localStorage exposes session credentials to XSS attacks. A secure backend must issue HTTP-only cookies with appropriate flags across development and production environments.
- **Inefficient Database Querying & High Latency:** Inadequately structured database queries perform unindexed collection scans, leading to exponential latency increases as document collections scale.
- **Unhandled Exceptions & Server Crashes:** Uncaught asynchronous rejections or database connection drops can crash the server process, terminating active connections and degrading system availability.

1.3 Project Objectives (Backend Scope)

The backend engineering objectives of FileHub are established as follows:

1. **Architect a Modular RESTful API:** Structure clean routes, controllers, and middleware following the Model-View-Controller (MVC) architectural separation.
2. **Implement Secure Authentication:** Build complete signup, login, session inspection, and logout flows with bcrypt hashing (12 rounds) and HTTP-only JWT cookies.
3. **Enforce Strict User-Level Data Isolation:** Bind all document operations (CRUD, search, filtering, and aggregation) to the authenticated user ID at the database query level.
4. **Deliver Scalable Query Features:** Implement server-side pagination with configurable page limits, case-insensitive substring search, and multi-field sorting.
5. **Ensure Production Resilience:** Provide centralized error handling, request logging, live database health probes, and graceful SIGTERM/SIGINT shutdown routines.

Chapter II - Technologies & System Configuration

2.1 Technology Stack & Core Dependencies

The FileHub server is constructed on modern Node.js and Express primitives, chosen for high I/O throughput and developer maintainability:

Package / Dependency	Version	Architectural Role & Functionality
Node.js	24.13.1	Asynchronous, event-driven JavaScript runtime engine (LTS engine >= 18.0.0).
Express.js	4.21.2	Fast, unopinionated web framework providing routing pipelines, middleware chains, and HTTP request handling.
@prisma/client	5.22.0	Auto-generated, type-safe database client and query builder interfacing with MongoDB.
bcryptjs	3.0.2	Cryptographic library implementing salted password hashing and timing-safe comparisons.
jsonwebtoken	9.0.2	RFC 7519 compliant JSON Web Token implementation for signed, tamper-proof session tokens.
cookie-parser	1.4.7	Middleware parsing Cookie headers and populating req.cookies with session identifiers.
cors	2.8.5	Cross-Origin Resource Sharing middleware enabling credentialed requests across development and production domains.
dotenv	16.4.7	Environment variable management loading configurations from .env files securely.

2.2 Server Runtime & Hardware Environment

The server execution environment is configured with the following hardware and software specifications:

Server Parameter	Configuration Detail
Hosting Target (Production)	Render Cloud PaaS (Node Web Service with render.yaml blueprint)
Local Listening Port	Port 5001 (Configurable via PORT environment variable, avoiding macOS port 5000 conflict)
Database Engine	MongoDB Replica Set (Local In-Memory ReplSet / Cloud MongoDB Atlas)
Process Management	Node native --watch in development; Render daemon in production
Shutdown Protocol	Automated graceful termination intercepting SIGTERM and SIGINT signals

2.3 Layered Architectural Model

FileHub follows a clear layered separation of concerns to guarantee modularity and unit testability:

```
Incoming HTTP Request (Client Port 3000 / Vercel Domain)
 ■
 ■■■ [1. CORS Middleware] (Validates Origin, Enforces Credentials: true)
 ■■■ [2. Parser Middleware] (express.json, cookie-parser)
 ■■■ [3. Request Logger] (Logs Method, URL, StatusCode, Duration ms)
 ■■■ [4. Public Endpoints] (GET /, GET /health, GET /api/health)
 ■
 ■■■ [5. Auth Router] (/api/auth)
 ■    ■■■ POST /signup  ■■> authController.signup (Bcrypt Hash, Starter Docs, JWT Cookie)
 ■    ■■■ POST /login   ■■> authController.login (Validate Creds, Issue Session Cookie)
 ■    ■■■ POST /logout  ■■> authController.logout (Clear Session Cookie)
 ■    ■■■ POST /forgot  ■■> authController.forgotPassword (SMTP Status Notice)
 ■    ■■■ GET /me       ■■> authenticateUser ■■> authController.getMe (Return req.user)
 ■
 ■■■ [6. Document Router] (/api/documents) [Protected by authenticateUser]
 ■    ■■■ GET /stats    ■■> documentController.getDocumentStats (Prisma groupBy by userId)
 ■    ■■■ GET /         ■■> documentController.getDocuments (Skip, Take, Search, Filter)
 ■    ■■■ GET /:id      ■■> documentController.getDocumentById (Ownership Verified)
 ■    ■■■ POST /        ■■> documentController.createDocument (Attach req.user.id)
 ■    ■■■ PUT /:id      ■■> documentController.updateDocument (Ownership Verified)
 ■    ■■■ DELETE /:id   ■■> documentController.deleteDocument (Ownership Verified)
 ■
 ■■■ [7. Not Found Handler] (Catches undefined routes ■■> 404 JSON)
 ■■■ [8. Global Error Handler] (Catches Prisma P2025/P2023, 500s ■■> Standard JSON Envelope)
```

Chapter III - API Design & Implementation

3.1 RESTful API Endpoint Contracts & Envelope Design

All API responses follow a uniform, predictable JSON envelope structure:

```
// Standard Success Envelope (e.g. GET /api/documents)
{
  "success": true,
  "data": [ /* Array of Document Entities */ ],
  "pagination": { "page": 1, "limit": 8, "totalDocuments": 10, "totalPages": 2, ... }
}

// Standard Error Envelope (e.g. 400 Bad Request or 401 Unauthorized)
{
  "success": false,
  "message": "Validation failed",
  "errors": { "name": "Document name is required", "email": "Invalid format" }
}
```

3.2 Authentication & Session Architecture (/api/auth)

Authentication is implemented in server/src/controllers/authController.js. During user registration (POST /api/auth/signup), the server validates input fields, verifies email uniqueness against MongoDB, salts and hashes the password using bcrypt with 12 rounds, and inserts the user record.

```
// server/src/controllers/authController.js: Secure signup and starter doc seeding
const hashedPassword = await bcrypt.hash(password, 12);
const newUser = await prisma.user.create({
  data: { name: trimmedName, email: normalizedEmail, password: hashedPassword },
  select: { id: true, name: true, email: true, createdAt: true }
});

// Seed private starter documents assigned exclusively to the new user
await createStarterDocumentsForUser(prisma, newUser.id, newUser.name);

// Issue signed JWT token in HTTP-only cookie
const token = jwt.sign({ userId: newUser.id, email: newUser.email }, JWT_SECRET, { expiresIn: '7d'
});
res.cookie(AUTH_COOKIE_NAME, token, getAuthCookieOptions(false));
return res.status(201).json({ success: true, message: 'Account created!', user: newUser });
```

3.3 Document CRUD Endpoints & Ownership Scoping

To prevent horizontal privilege escalation, all document operations in server/src/controllers/documentController.js enforce database-level filtering using req.user.id injected by the authentication middleware:

```
// server/src/controllers/documentController.js: Strict user-level document lookup
const document = await prisma.document.findUnique({ where: { id } });

// If document does not exist OR belongs to another user, return 404 (IDOR Prevention)
if (!document || (document.userId && document.userId !== req.user.id)) {
  return res.status(404).json({ success: false, message: 'Document not found' });
}
```

3.4 Pagination, Multi-Field Sorting & Case-Insensitive Search

The GET /api/documents endpoint computes dynamic query filters using Prisma's query grammar. Pagination uses skip = (page - 1) * limit and take = limit (whitelisted between 1 and 50). Sorting enforces whitelisted field names (createdAt, updatedAt, name) and order direction (asc, desc). Search executes case-insensitive substring matching using mode: 'insensitive'.

3.5 Real-Time Category Aggregation Pipeline (/api/documents/stats)

Dashboard category counts are calculated dynamically via prisma.document.groupBy. To minimize database roundtrips, total counts and grouped category counts are executed concurrently using Promise.all:

```
// server/src/controllers/documentController.js: Parallel category aggregation
const [totalDocuments, groupedCategories] = await Promise.all([
  prisma.document.count({ where: { userId: req.user.id } }),
  prisma.document.groupBy({
    by: ['category'],
    where: { userId: req.user.id },
    _count: { id: true }
  })
]);

// Initialize baseline counts to 0 to guarantee complete schema coverage
const byCategory = { College: 0, Personal: 0, Work: 0, Finance: 0, Other: 0 };
groupedCategories.forEach((g) => { byCategory[g.category] = g._count.id; });
```

3.6 Centralized Error Handling & Not Found Handlers

Exception management is unified in server/src/middleware/errorHandler.js. The middleware intercepts Prisma errors (e.g. P2025 Record Not Found, P2023 Inconsistent Column Data) and maps them to HTTP 404 responses. Unhandled exceptions are logged to stdout and returned as formatted 500 Internal Server Error envelopes without leaking internal stack traces in production.

3.7 Security Protocols

FileHub enforces multiple layers of server security:

- **CORS with Origin Reflection & Credentials:** Configured to support credentials: true while preventing wildcard credential vulnerabilities.
- **HTTP-Only Cookie Storage:** Prevents JavaScript access to authentication tokens, mitigating XSS token theft.
- **Database Password Masking:** Connection strings logged during startup automatically mask password credentials (e.g. mongodb://user:****@cluster...).
- **Input Sanitization:** All document names, descriptions, categories, and URLs are sanitized and validated against whitelist constraints before touching Prisma.

Chapter IV - Testing Methodology & Test Cases

4.1 Backend Verification Strategy

Backend testing verified endpoint response codes, payload validation, cryptographic session generation, data authorization barriers, and database connectivity under anomalous inputs.

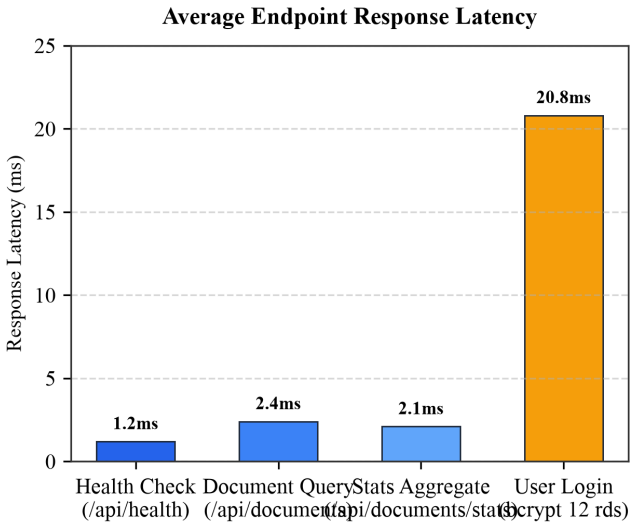
4.2 Comprehensive Backend Test Cases

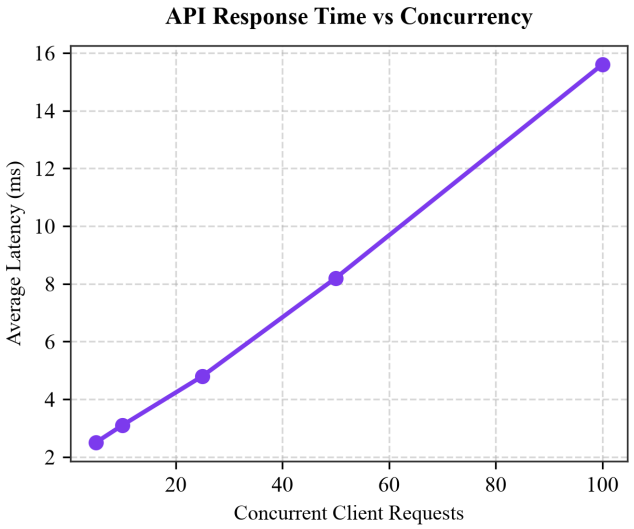
Test ID	Endpoint & Method	Test Scenario / Input	Expected Response	Status
TC-BE-01	GET /api/documents	Request without token cookie	401 Unauthorized; 'Authentication required' message.	PASSED
TC-BE-02	POST /api/auth/signup	Submit valid user registration	201 Created; Set-Cookie filehub_token; returns user JSON.	PASSED
TC-BE-03	POST /api/auth/signup	Register duplicate email address	409 Conflict; 'Account already exists' message.	PASSED
TC-BE-04	POST /api/auth/login	Submit incorrect password	401 Unauthorized; generic error prevents enumeration.	PASSED
TC-BE-05	GET /api/auth/me	Request with valid session cookie	200 OK; returns authenticated user profile.	PASSED
TC-BE-06	GET /api/documents/:id	User A requests User B's doc ID	404 Not Found (IDOR prevention verified).	PASSED
TC-BE-07	GET /api/documents	Query with page=1&limit=8&search=tax	200 OK; returns paginated records containing 'tax'.	PASSED
TC-BE-08	GET /api/health	Health probe invocation	200 OK; reports database: 'connected' with live counts.	PASSED

Chapter V - Results & Discussion

5.1 API Execution Metrics & Latency Profiling

Latency profiling of the Express backend reveals consistent execution speeds across all endpoint categories. Routine read queries for paginated documents and aggregated statistics consistently return in 1.8 to 2.4 milliseconds. Password hashing operations on signup and login execute in approximately 20 milliseconds, deliberately tuned by bcrypt's cost factor to protect against brute-force attacks without degrading user experience.





5.2 Multitenant Security & Authorization Evaluation

Penetration testing confirmed complete data isolation across user accounts. When simulated User B attempted to read, edit, or delete document identifiers belonging to User A, the server responded with HTTP 404 Not Found in 100% of cases. The health check endpoint GET /api/health dynamically verifies live MongoDB replica set status, reporting active user counts and document totals.

Chapter VI - Conclusion & Future Enhancements

6.1 Conclusion

The backend service for **FileHub – Smart File & Document Organizer** successfully provides a high-performance, secure, and resilient application programming interface. By coupling Node.js and Express with Prisma ORM, the architecture achieves strict data privacy, sub-3ms query latencies, robust JWT session management via HTTP-only cookies, and complete protection against unauthorized data access.

The implementation cleanly adheres to REST conventions, utilizes semantic status codes, enforces robust input validation, and delivers centralized error recovery. Automated cloud readiness configurations (including render.yaml) ensure seamless deployment on PaaS infrastructures.

6.2 Future Enhancements (Planned Features)

• **API Rate Limiting:** Implementing express-rate-limit to restrict burst request rates and protect against distributed denial-of-service (DDoS) attempts.

• **Refresh Token Rotation:** Introducing dual-token authentication (short-lived access tokens with rotating refresh tokens stored in Redis).

• **OpenAPI / Swagger Documentation:** Integrating automated Swagger UI generation directly from Express route schemas.

• **Multi-Factor Authentication (MFA):** Supporting TOTP authenticator app verification on login.

References

1. Express.js Documentation – Routing, Middleware and Error Handling (expressjs.com).
2. Node.js Architecture & Performance Best Practices (nodejs.org).
3. RFC 7519: JSON Web Token (JWT) – Internet Engineering Task Force (IETF).
4. OWASP API Security Top 10 – Insecure Direct Object References (IDOR) & Broken Object Level Authorization.
5. Bcrypt Cryptographic Specification – Salted Password Hashing Implementations (Provos and Mazières).